Real Estate Pro

Техническое задание

Версия 1.1, исправленная редакция от 27.10.2026.

Курс: «Проектирование и разработка архитектуры сервисов и приложений».
Команда: Даниил Матвеев, Илья Голубев, Дима Будаев, Тимур Гаджигасанов.

1. Общие сведения

1.1. Назначение. Агрегатор объявлений об аренде и продаже квартир: сбор из подключённых источников, нормализация и поиск с фильтрами. Пользователь сохраняет поиск и получает Telegram-уведомления о подходящих новых объявлениях. Международный поиск демонстрируется на синтетических данных; охват реального источника определяется его API.

1.2. Цель. Три микросервиса: core-api, collector, notifier; PostgreSQL; один Redis для кэша, дедупликации и Streams; развёртывание через Docker Compose; простой веб-интерфейс.

1.3. Аудитория и роли. Посетитель ищет без регистрации; подписчик управляет поисками через Telegram; администратор запускает служебные операции с ключом доступа.

1.4. Источники и ограничения

Реальные данные запрашиваются только через разрешённые официальным API операции с необходимыми правами и ограничением частоты. В проекте HTML-парсинг площадок не используется. Возможность получить объявления через конкретный API, способ авторизации, тариф и доступные категории проверяются по документации выбранного метода; универсальные требования к оплате или передаче логина в каждом запросе не предполагаются.

Хранятся нормализованные поля и ссылка на первоисточник. Объём хранения текстов и ссылок на изображения определяется условиями источника. Синтетические объявления явно помечаются как mock; их ссылки не выдаются за реальные объявления.

Демонстрация не требует платного аккаунта или сетевого доступа к площадке. Реальный адаптер Авито реализуется по документированному методу, возвращающему необходимые поля, и проверяется на фикстурах. Проверка с площадкой проводится только при доступных правах. Реквизиты метода и ограничения фиксируются в docs/contracts.md; тест на фикстурах не считается проверкой реального доступа.

Из пользовательских данных хранятся Telegram chat_id, параметры подписок и технические записи доставки. Команда /stop удаляет пользователя, подписки и связанные записи; payload объявлений в стриме не содержит chat_id.

2. Функциональные требования: Collector

Collector содержит адаптеры, нормализатор, конвейер доставки и веб-маршруты. Реальный и mock-источник проходят один путь: адаптер → RawListing → нормализация → предварительная дедупликация → Core API.

ФТ-1.1. Минимум два адаптера: AvitoApiAdapter и MockAdapter. Общий интерфейс fetch() → list[RawListing]. Адаптер скрывает пагинацию источника. RawListing - внутренний общий формат исходных полей; ответ внешнего API преобразуется в него внутри адаптера. Схемы ответа API, RawListing и нормализованного запроса описываются отдельно в docs/contracts.md.

ФТ-1.2. MOCK_MODE=true по умолчанию включает локальные данные. При MOCK_MODE=false нужны настройки доступа. При их отсутствии сервис явно сообщает об ошибке и переходит на mock; при отказе API, таймауте или блокировке fallback применяется к текущему источнику. Подмена отражается в логах и поле source=mock; реквизиты реальных объявлений не присваиваются синтетическим.

ФТ-1.3. Нормализация: цена в USD с сохранением исходной цены и валюты, очистка текста от HTML, комнаты целым числом (студия=0), площадь в м², страна и город по справочникам. Неполные и некорректные записи пропускаются с причиной в логе. Правила чисел, округления и справочников - в разделе 3.

ФТ-1.4. Пакетный сбор запускается по расписанию (15 минут по умолчанию) и вручную через POST /collect/run. В mock-режиме ручной запуск обрабатывает весь дамп, сохраняя внешние ID. Эндпоинт с ключом X-Admin-Key ставит задачу в фон и возвращает 202 с run_id; повторный запуск при активной задаче возвращает 409. Статус: GET /collect/runs/{run_id}, также с ключом. Параллельные циклы одного источника не допускаются.

ФТ-1.5. Redis ускоряет проверку дублей; окончательная проверка выполняется Core API и PostgreSQL. Хэш отмечается обработанным только после успешного ответа Core API. При недоступности Redis отправка продолжается через идемпотентный API.

ФТ-1.6. Доставка: POST /api/v1/listings; таймауты и до пяти повторов после начальной попытки при сетевой ошибке, 429 или 5xx, с экспоненциальной задержкой 1-30 с и учётом Retry-After. Остальные 4xx не повторяются; после исчерпания попыток запись остаётся доступной для повторного запуска. Внешний ID при повторе не меняется.

ФТ-1.7. Две страницы Jinja2 + HTMX: список с фильтрами и карточка со ссылкой на источник. Обращения к данным идут через Core API; прямого доступа к PostgreSQL нет. Для mock показывается явная отметка о синтетических данных.

ФТ-1.8. Потоковый mock-генератор отправляет по одной новой записи через случайные интервалы 5-30 с. Режим задаётся MOCK_PUBLISH_MODE=stream либо batch (по умолчанию batch). В stream планировщик пакетного сбора отключён, ручная загрузка всего дампа остаётся доступной. Новый объект получает уникальный external_id; повтор доставки того же объекта сохраняет его ID. Внешние ID дампа стабильны между импортами; новые сгенерированные ID не пересекаются с ними.

2.2. Core API и 2.3. Notifier

Core API

ФТ-2.1. POST /api/v1/listings валидирует и сохраняет объявление. Новая запись: 201 {id, created:true}; дубль: 200 {id, created:false}. При дубле поля существующей записи не меняются, новое событие не создаётся. Параллельные запросы защищаются уникальным ограничением БД.

ФТ-2.2. GET /api/v1/listings: deal_type, country, city, price_min/price_max в USD, rooms, area_min/area_max, source, sort, page, size. sort: price_asc, price_desc, date_asc, date_desc; по умолчанию date_desc. При равенстве используется id. page≥1, size=20 по умолчанию, максимум 100; некорректные диапазоны дают 422. Ответ: {items, total, page, size}.

ФТ-2.3. GET /api/v1/listings/{id} возвращает карточку; неизвестный ID даёт 404. ФТ-2.4. GET /api/v1/countries, /api/v1/cities, /api/v1/sources, /api/v1/currencies возвращают справочники. Города фильтруются по country. Форматы элементов фиксируются в contracts.md.

ФТ-2.5. Для нового объявления создаётся событие listings.new. Запись объявления и задания публикации атомарна в PostgreSQL (transactional outbox, раздел 4.3); отдельная фоновая задача Core API выполняет XADD. Успешный HTTP-ответ означает фиксацию записи и задания, а не завершённую отправку в Telegram.

ФТ-2.6. Списки кэшируются; новые записи инвалидируют кэш по разделу 3.5.
ФТ-2.7. GET /health возвращает status и checks с состоянием postgres и redis (ok или unavailable). Обе зависимости доступны: 200 ready. Недоступен только Redis: 200 degraded. Недоступен PostgreSQL: 503 unavailable. При отказе Redis сохраняются чтение из БД и запись с outbox.
GET /health/live возвращает 200 alive без обращения к зависимостям. GET /health/ready возвращает 200 ready при доступности обеих зависимостей и 503 при отказе любой из них. OpenAPI доступен в /docs.

Notifier

ФТ-3.1. /start регистрирует; /subscribe создаёт подписку (тип сделки, страна, город, комнаты, максимальная цена в USD); /list показывает подписки; /unsubscribe <id> удаляет только свою; /stop удаляет пользователя и его данные. Страна и город проверяются по справочникам Core API; при их недоступности используются ранее загруженные данные, иначе создание подписки отклоняется с понятным сообщением.

ФТ-3.2. Пользователи и подписки хранятся в схеме notifier. ФТ-3.3. Чтение listings.new через consumer group notifier и пул из восьми воркеров; подтверждение и восстановление pending описаны в 4.3.

ФТ-3.4. Совпадение требует выполнения всех заданных критериев. Несколько подписок одного пользователя дают одно уведомление с ценой, адресом (если есть), комнатностью и ссылкой.

ФТ-3.5. Уникальный журнал по (user_id, listing_id) предотвращает повторную отправку уже подтверждённого уведомления. Параллельные воркеры атомарно резервируют доставку. Сбой между ответом Telegram и записью результата может оставить неопределённый статус; абсолютное exactly-once внешней доставки не заявляется. Правила повторов - в 4.3.

ФТ-3.6. Без TELEGRAM_BOT_TOKEN совпадения пишутся в stdout; режим доставки log хранится отдельно от telegram и не блокирует будущую реальную отправку. ФТ-3.7. GET /health проверяет зависимости; POST /debug/event с X-Admin-Key валидирует payload и добавляет его в тот же stream, не создавая объявление в core. Для демонстрации используются отдельные тестовые ID.

3. Бизнес-правила

3.1. Объявление и валидация

Обязательны: source, external_id, url, deal_type (rent/sale), country, city, price, currency, price_usd, rooms. rooms - целое ≥0, null недопустим. Необязательны: title, address, area_m2, floor, description, photos (URL), published_at. Отсутствующие необязательные значения - null либо поле отсутствует по контракту. area_m2, если задана, >0; денежные суммы >0. Ошибка обязательного поля или справочника: 422 {error, message}.

3.2. Валюты и время

Курс currencies.rate_to_usd означает стоимость одной единицы валюты в USD; USD=1. Collector получает курсы через Core API; исходные значения задаются миграциями и не меняются в ходе демонстрации. price_usd = price × rate_to_usd с округлением ROUND_HALF_UP до целого USD. Вычисления Decimal, в PostgreSQL NUMERIC; JSON использует числа. Core API проверяет результат по тому же курсу. Даты передаются в ISO 8601 UTC; Europe/Minsk используется для отображения и расписания.

3.3. Дедупликация

Тождество объявления определяется парой (source, external_id), обязательной в API. url_hash = SHA-256 от однозначной сериализации этой пары; имя поля историческое, это хэш идентичности. В PostgreSQL - UNIQUE(source_id, external_id) и UNIQUE(url_hash). Нормализованный URL используется адаптером для стабильного external_id только если источник не предоставляет ID. Алгоритм сериализации фиксируется в contracts.md.

Redis SET dedup:urls хранит хэши успешно принятых записей и может быть восстановлен или очищен без потери объявлений. Истёкшая/потерянная запись Redis не нарушает уникальность БД; сброс core при разработке требует сброса dedup:urls.

content_hash строится из города с учётом страны, адреса, комнат, площади и типа сделки. Совпадение у разных источников создаёт связь «возможный дубль»; обе записи сохраняются. При неполных данных автоматическое связывание не выполняется. Такая связь не является доказательством тождества квартиры.

3.4. Mock и источники

Обязателен воспроизводимый синтетический дамп: ≥500 объектов, ≥5 стран, ≥10 городов; реалистичные цены и комнатность. Дамп хранит внутренний RawListing; отдельные фикстуры ответа Авито проверяют преобразование AvitoApiAdapter. Международные синтетические данные не объявляются реальным ответом Авито. Из дампа доступны массовая загрузка и потоковая генерация по его образцам.

3.5. Кэш

Ключ списка включает хэш нормализованных фильтров, сортировки и пагинации и версию кэша; TTL 600 с. Новая запись инициирует увеличение cache:version. При отказе Redis используется БД; после восстановления доступ к старому кэшу возобновляется только после успешного увеличения версии. Событие инвалидации сохраняется вместе с outbox-заданием. Незавершённая инвалидация не должна позволять возвращать устаревший список после восстановления.

3.6. Параметры и 3.7. Справочники

Параметр	Значение по умолчанию
TTL кэша	600 с
Страница: размер / максимум	20 / 100
Пакетный сбор	15 минут; не запускается по расписанию в stream-режиме
Интервал mock-потока	Случайный, от 5 до 30 с
Повторы Core API	До 5 после первой попытки; задержки 1-30 с
Подписки пользователя	До 10
Telegram-уведомления	До 30 на пользователя за скользящий час
Воркеры notifier	8
Порог контроля длины stream	10 000 событий; мягкий порог, без удаления необработанных
Pending до повторного захвата	30 с; продление владения при долгой обработке
Режим mock по умолчанию	MOCK_MODE=true; MOCK_PUBLISH_MODE=batch
Часовой пояс	Хранение и обмен: UTC; отображение: Europe/Minsk

Численные параметры настраиваются; изменения относительно значений ТЗ отражаются в инструкции запуска. При превышении лимита уведомлений задания сохраняются в PostgreSQL и откладываются; не отбрасываются и не удерживают pending-событие до истечения часа.

3.7. Справочники стран и городов

countries: код ISO 3166-1 alpha-2 в верхнем регистре и название. cities: ссылка на страну, каноническое латинское название и альтернативные написания. Пример: BY / Minsk; альтернативы «Минск», «Мінск». Уникальность города задаётся внутри страны; сопоставление только по названию без страны не допускается.

Справочники заполняются миграциями. Collector нормализует пробелы и регистр при поиске альтернативы; API, фильтры, события и подписки используют каноническое название и код страны. Неизвестные и неоднозначные значения дают 422; автоматическое создание городов из входящего объявления не выполняется. Отображаемое локализованное название не меняет каноническое значение.

4. Архитектура: состав

Сервис	Стек / порт	Ответственный
core-api	Python, FastAPI, SQLAlchemy, Alembic / 8000	Тимур; БД: Илья
collector	Python, httpx, Jinja2, HTMX / 8001	Тимур; UI и данные: Дима
notifier	Go, go-redis, Telegram Bot API / 8080	Даниил
postgres	PostgreSQL, схемы core и notifier / 5432	Илья; Compose: Даниил
redis	Один Redis 7+, кэш, SET, Streams / 6379	Интеграция: Даниил

4.2. Данные и 4.3. Доставка событий

4.2. PostgreSQL

core: countries, cities, альтернативные названия городов (таблица либо поле по решению Ильи), sources, currencies, listings; связи listing_duplicates для возможных дублей; outbox_events для публикации и инвалидации. listings ссылается на город, источник и валюту; поля - по 3.1. Индексы: (deal_type, city_id, price_usd), rooms, created_at DESC и ограничения уникальности из 3.3.

notifier: users с уникальным chat_id, subscriptions, notifications_log с уникальной парой пользователь/объявление и состоянием доставки; задания отложенной отправки с адресатом и режимом log/telegram. ID объявления - внешнее значение из события, без FK в core. Сервисы используют только свою схему, остальные данные получают через API/события. /stop удаляет и отложенные задания; перед отправкой проверяется существование пользователя.

4.3. Redis Streams и outbox

Публикация. В одной транзакции PostgreSQL создаются объявление и outbox-задание с устойчивым event_id. Фоновый обработчик core-api выполняет XADD listings.new * data <json>, затем отмечает публикацию. Ошибка Redis оставляет задание для повтора. Повтор после сбоя может создать несколько записей stream с одним event_id; потребитель обязан учитывать это. Дубль POST не создаёт нового outbox-задания.

Чтение. Группа создаётся идемпотентно командой XGROUP CREATE listings.new notifier 0 MKSTREAM, чтобы прочитать и ранее записанные события. Каждый экземпляр имеет уникальное имя consumer. Новые записи читаются XREADGROUP, pending старше 30 с - XAUTOCLAIM. При обработке дольше порога владение продлевается или порог увеличивается.

Подтверждение. После сопоставления подписок задания доставки атомарно сохраняются в notifier с уникальными ключами. Затем выполняется XACK; отправка Telegram идёт по сохранённым заданиям. Событие без совпадений также подтверждается. Временные ошибки БД/Redis повторяются. Некорректный payload сохраняется в listings.dead с причиной и ID исходной записи, и лишь затем подтверждается.

Telegram. Уже записанное состояние sent не отправляется повторно; два воркера не могут одновременно владеть одним заданием. Для 429 учитывается Retry-After, постоянные ошибки сохраняются с причиной. При таймауте или падении после отправки, но до фиксации результата, статус становится unknown и требует явного решения о повторе; автоматическая exactly-once доставка через Telegram не гарантируется.

Хранение. Redis использует volume, AOF (appendfsync everysec) и noeviction. Порог 10 000 - сигнал контроля; безусловный MAXLEN при XADD запрещён. Очистка удаляет только уже подтверждённые события за безопасной границей, учитывая pending и ещё не прочитанные записи. До реализации безопасной очистки stream не обрезается. Перезапуск notifier не теряет записи; авария Redis может потерять последний интервал AOF. Сбой диска и удаление volume не покрываются гарантией.

4.3. Контракт события и устройство Collector

Каждая запись listings.new содержит поле data с JSON. event_id - устойчивый UUID задания outbox; id - положительный целочисленный ID объявления; source - строка; deal_type - rent/sale; country - ISO alpha-2; city - каноническая строка; rooms - целое ≥0; price и price_usd - положительные числа; currency - строка ISO 4217; address - строка либо null; url - строка URL; created_at - ISO 8601 UTC. Все ключи присутствуют. area_m2 в событие не входит: фильтра по площади в подписках нет. title и description не обязательны для notifier и не входят в базовый payload.

Пример data:

{"event_id":"550e8400-e29b-41d4-a716-446655440000",
"id":123, "source":"mock", "deal_type":"rent",
"country":"BY", "city":"Minsk", "rooms":1,
"price":380, "currency":"USD", "price_usd":380,
"address":"ул. Примерная, 1",
"url":"https://example.com/listing/123",
"created_at":"2026-10-27T10:00:00Z"}

Названия и типы фиксируются в README и docs/contracts.md. При расхождениях приоритет у этого ТЗ; изменения согласуются с владельцами затронутых сервисов. Все значения примера синтетические.

4.4. Поток данных

Collector → POST /api/v1/listings → транзакция core (объявление + outbox) → фоновый XADD → Redis Stream listings.new → notifier → сохранённые задания доставки → Telegram. Веб-интерфейс collector читает Core API. Отдельный сервис для outbox не требуется.

4.5. Устройство Collector

AvitoApiAdapter преобразует документированный ответ API в RawListing. MockAdapter читает RawListing из локального дампа. Общий нормализатор создаёт запрос Core API. В batch данные обрабатываются списком; в stream генератор по одному создаёт новые объекты. Сетевой клиент имеет таймауты и ограничение частоты. API площадки не предполагается глобальным поиском по всем объявлениям, если это не подтверждено выбранным методом.

4.6. Необязательный mock-сервер источника

Дополнительный контейнер хранит синтетические данные и по таймеру делает новые записи доступными через HTTP. Collector опрашивает его; сервер сам не пишет в Core API или stream. Для совместимости реализуются нужные методы, пагинация, ответы и заглушка авторизации выбранного API; одного изменения URL недостаточно без совпадения этого контракта. Учётные данные реального API mock-серверу не передаются. Подключение - отдельным Compose-профилем; без него работают обязательные встроенные моки.

5. Нефункциональные требования

5.1. Безопасность

НФТ-1. Токен Telegram, пароли PostgreSQL, ключ администратора и реквизиты API задаются через .env; в Git только .env.example с пустыми/учебными значениями. Секреты исключаются из логов. НФТ-2. SQL параметризован или выполняется ORM; Pydantic валидирует входы Python API, Go-сервис выполняет эквивалентную проверку своих входов.

НФТ-3. /debug/*, /collect/run и статусы заданий доступны с X-Admin-Key; ключ задаётся конфигурацией. НФТ-4. PostgreSQL и Redis доступны во внутренней сети Compose, публикация портов - только для разработки.

5.2. Надёжность

НФТ-5. PostgreSQL и Redis используют volumes. Перезапуск/пересоздание контейнеров без удаления volumes сохраняет данные в пределах гарантий БД и AOF. НФТ-6. Структура меняется миграциями: Alembic core, SQL notifier; миграции применяются до готовности сервисов, без ручного создания таблиц.

НФТ-7. При отказе Redis Core API читает из БД и сохраняет outbox; collector использует повторы при недоступности API; notifier восстанавливает pending и задания из БД. Возобновление кэша - по 3.5. НФТ-8. Healthcheck и depends_on задают начальный порядок запуска; сервисы также умеют переподключаться при сбое после запуска.

НФТ-9. Внешние вызовы ограничены таймаутами; ошибки и завершение сервиса не оставляют бесконечно зависших запросов. Pending, отложенные задания, outbox и ошибки публикации отражаются в логах/состоянии сервиса.

5.3. Производительность

НФТ-10. Для 500+ объявлений p95 времени GET списка: <300 мс с прогретым кэшем, <500 мс без кэша, при одном последовательном клиенте; 100 запросов после прогрева. В отчёте фиксируются машина, состав данных и фильтры; сетевое время до внешних площадок не учитывается.

НФТ-11. При доступных зависимостях, свободном лимите пользователя и потоке не чаще 1 события за 5 с p95 задержки от фиксации нового объявления до успешного ответа Telegram <5 с. В log-режиме измеряется запись совпадения. Проверка минимум на 100 событиях. Время аварии, отложенной доставки по лимиту и ожидания Retry-After не входит; эти задержки учитываются отдельно.

5.4. Разработка и сопровождение

НФТ-12. Ошибки JSON API: {error, message} с корректным кодом, в том числе 422 валидации. НФТ-13. После настройки .env система запускается docker compose up --build. НФТ-14. Git хранит исходники; README описывает статус реализации, запуск, контракты и настройки. Перед новой задачей выполняются fetch и просмотр изменений; работа ведётся в ветке задачи, дочерние ветки сливаются в неё, готовый этап - через PR в main.

5.5. Интерфейс

НФТ-15. Страницы списка и карточки работают в актуальных браузерах; ошибки API и отсутствие результатов отображаются понятно пользователю.

6. Распределение работ

6.1. Илья Голубев: PostgreSQL и сопровождение

Обе схемы: таблицы, связи, ограничения, индексы и миграции; справочники стран, городов, альтернативных написаний и валют; ER-диаграмма, роли и права, анализ запросов, резервное копирование и проверка восстановления. Совместно с разработчиками согласует outbox, связи возможных дублей и состояния/задания доставки. Структура изменяется только через миграции.

6.2. Дима Будаев: интерфейс, данные, ручные проверки

По готовым примерам оформляет HTML/CSS и Jinja2-шаблоны списка и карточки; готовит примеры и проверяет синтетический RawListing-дамп (500+ объектов, 5 стран, 10 городов), чек-лист, сообщения об ошибках, инструкцию запуска и демонстрацию. Тимур предоставляет формат, шаблон и генератор большого набора; Дима не отвечает за авторизацию API, парсеры и асинхронную доставку.

6.3. Тимур Гаджигасанов: Python-бэкенд

Core API: валидация, запись, поиск, справочники, кэш, outbox и публикация Streams. Collector: реальный адаптер, mock, нормализация, дедупликация, batch/stream-режимы, повторы, расписание и веб-маршруты. При необходимости - mock-сервер. Модели и запросы согласуются с Ильёй, событие - с Даниилом.

6.4. Даниил Матвеев: Notifier и интеграция

Go-сервис, Telegram-команды, подписки, consumer group, pending, сопоставление, журнал, отложенные задания, повторы и лимиты доставки. Docker Compose, настройки, volumes PostgreSQL/Redis, AOF, healthcheck и сквозная интеграция. SQL-миграции notifier готовит Илья; применение при запуске согласуют вместе.

6.5. Границы ответственности

Илья владеет структурой БД; разработчик сервиса описывает изменения и обновляет модели/запросы. Сервисы не обращаются к чужой схеме; notifier хранит ID объявления без межсхемного FK. Тимур отвечает за публикацию и кэш, Даниил - за чтение, XACK, повторную обработку и доставку. Дима обновляет README, техническую точность проверяют владельцы сервисов. Контракты фиксируются до параллельной разработки, изменения согласуются затронутыми участниками.

7. Изменения редакции 1.1 от 27.10.2026

Сохранены решения версии 1.1: API-адаптер + mock, потоковая генерация, справочники, исключение area_m2 из события, Redis Streams. Уточнены общий RawListing, независимость синтетики от географии Авито, доступ к API, batch/stream и стабильные ID при повторах. Устранены необоснованные гарантии exactly-once Telegram и безусловной сохранности stream при MAXLEN. Добавлены outbox, безопасное хранение и очистка, состояния доставки, правила валют и проверяемые условия производительности. Разделы 6.6 и 6.7 не добавлялись.